Assignment 1

- Student: **Danilo Tambone**
- NetID: **PWC25003**

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Set display options to show all columns
pd.set_option('display.max_columns', None)
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
df = pd.read_csv('/content/drive/MyDrive/Predictive Modeling/Module 1/Telco_customer_churn.csv')
```

1.Read in the dataset and determine how many rows and columns it has.

```
df.shape
```

```
(7043, 33)
```

The dataset contains 7043 rows and 33 columns

2. When you open a dataset Python will assume the data types for each column. What are the current continuous variables?

```
# selecting only the numerical datatypes and visualizing only the first 5 rows.
print(df.select_dtypes(include=['int64', 'float64']).head())
```

```
   Count  Zip Code   Latitude   Longitude  Tenure Months  Monthly Charges  \
0      1     90003  33.964131 -118.272783              2            53.85
1      1     90005  34.059281 -118.307420              2            70.70
2      1     90006  34.048013 -118.293953              8            99.65
3      1     90010  34.062125 -118.315709             28           104.80
4      1     90015  34.039224 -118.266293             49           103.70

   Churn Value  Churn Score  CLTV
0            1           86  3239
1            1           67  2701
2            1           86  5372
3            1           84  5003
4            1           89  5340
```

The continuous variables in this dataset are Monthly Charges, Total Charges, Tenure Months, and CLTV. I have included Total Charges here because, despite being initially stored as an 'object' due to formatting issues, it represents quantitative monetary data. Latitude and Longitude are also numeric, though they serve as geographic identifiers. ZIP CODE is also categorical since this is not quantitative, despite being stored as an 'numeric'

3.What are the current categorical variables?

```
# Since in the datatypes wherever we see "object" it is a categorical variable, let's filter with this condition

print(df.select_dtypes(include=['object']).head(1))
```

```
   CustomerID        Country       State         City                Lat Long  \
0  3668-QPYBK  United States  California  Los Angeles  33.964131, -118.272783

  Gender Senior Citizen Partner Dependents Phone Service Multiple Lines  \
0   Male             No      No         No           Yes             No

  Internet Service Online Security Online Backup Device Protection  \
0              DSL             Yes           Yes                No

  Tech Support Streaming TV Streaming Movies        Contract  \
0           No           No               No  Month-to-month

  Paperless Billing Payment Method Total Charges Churn Label  \
0               Yes   Mailed check        108.15         Yes
```

```
                 Churn Reason
0  Competitor made better offer
```

Everyone of the columns above are categorical variables, excepting for Total Charges. Also binary indicators are categorical even if often shown as numerical, so I have to include the Churn Value column too. In my opinion, ZIP CODE is also categorical since this is not quantitative. Even if Lat Long appears as a number, it's a concatenation of the two numeric values, so it's stored like a string and that means categorical.

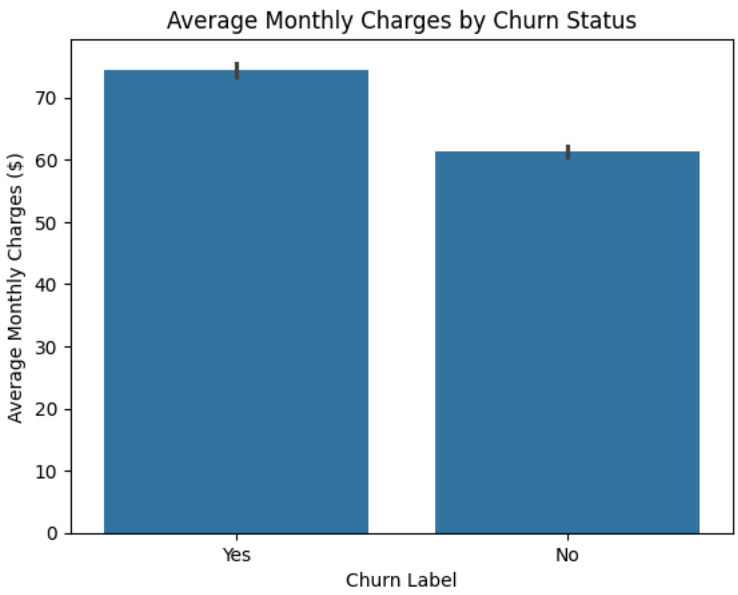
```
Start coding or generate with AI.
```

4. Are there any columns that look like they might be the wrong data type?

In my opinion, Total Charges is an incorrect data type because it is stored as an object but represents a continuous numeric value. Additionally, Zip Code should be a string rather than an integer, as it is a categorical identifier and not a quantitative measurement.

5. Build a bar chart to show average Monthly Charges by Churn Label. How do the monthly changes compare for those who churn and those who do not?

```
# Create a bar chart showing average Monthly Charges by Churn Label
sns.barplot(data=df, x='Churn Label', y='Monthly Charges')
# Selecting title and axis labels
plt.title('Average Monthly Charges by Churn Status')
plt.xlabel('Churn Label')
plt.ylabel('Average Monthly Charges ($)')
# Save the plot
plt.show()
```



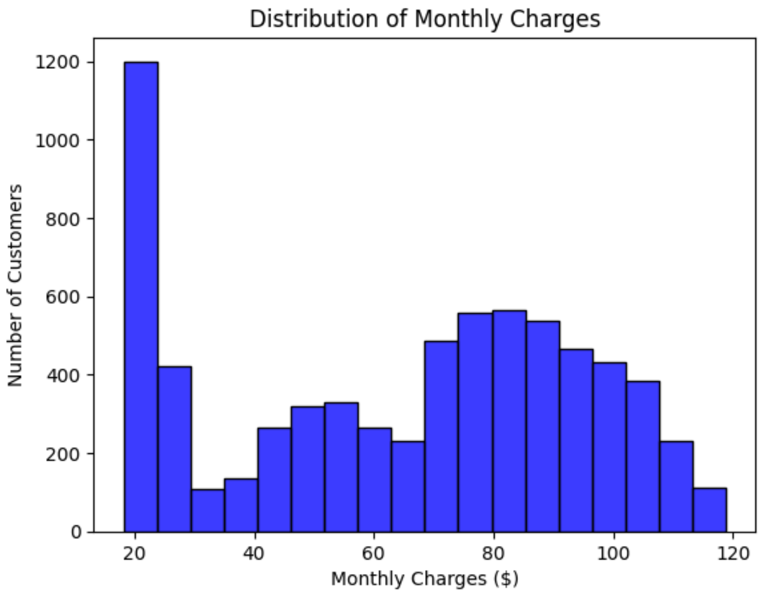
On average, monthly charges were higher for customers who left the service than for those who stayed.

6.Build a histogram of Monthly Charges. Describe some features of the distribution.

```
sns.histplot(data=df, x='Monthly Charges', color='blue')

# Adding labels for clarity
plt.title('Distribution of Monthly Charges')
plt.xlabel('Monthly Charges ($)')
plt.ylabel('Number of Customers')

plt.show()
```



The histogram shows a major peak at the 20 mark, meaning a large number of customers are grouped at the lowest price point. After a dip in the middle range (30–60), there is a second, wider peak between 70 and 100. This suggests the customer base is split between those choosing a basic, inexpensive option and those opting for more expensive services.

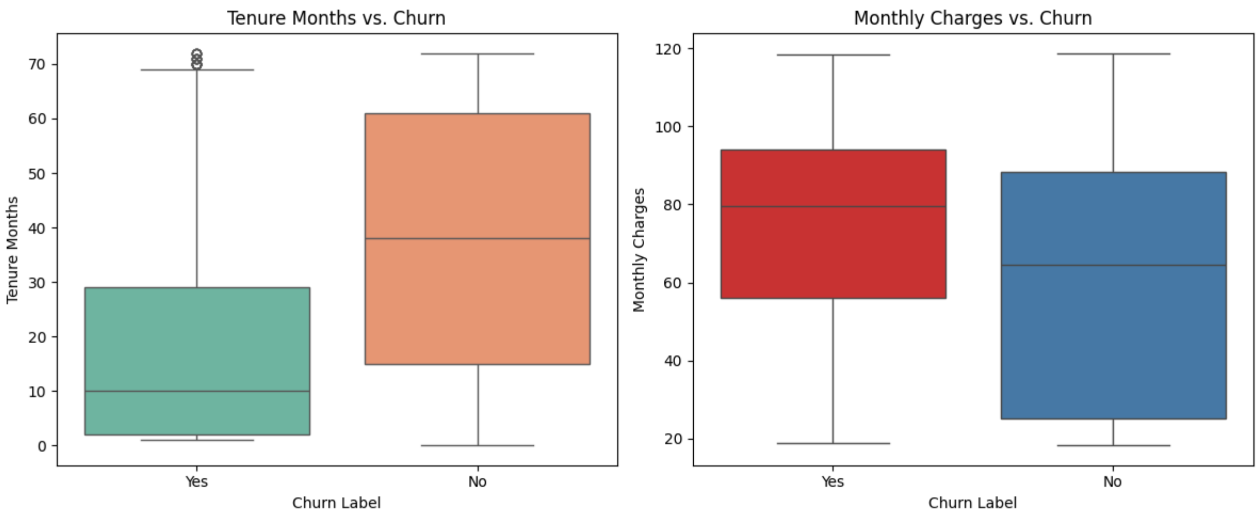
7. Make comparative boxplots for Tenure Months by Churn Label and Monthly Charges by Churn Label. Comment on what you see about Tenure Months and Monthly Charges as possible predictors of churn.

```
# Setting up a side-by-side view for both plots
plt.figure(figsize=(12, 5))

# Plot 1: Tenure Months by Churn Label
plt.subplot(1, 2, 1)
sns.boxplot(data=df, x='Churn Label', y='Tenure Months', hue='Churn Label',palette='Set2')
plt.title('Tenure Months vs. Churn')

# Plot 2: Monthly Charges by Churn Label
plt.subplot(1, 2, 2)
sns.boxplot(data=df, x='Churn Label', y='Monthly Charges', hue='Churn Label', palette='Set1')
plt.title('Monthly Charges vs. Churn')

plt.tight_layout()
plt.show()
```



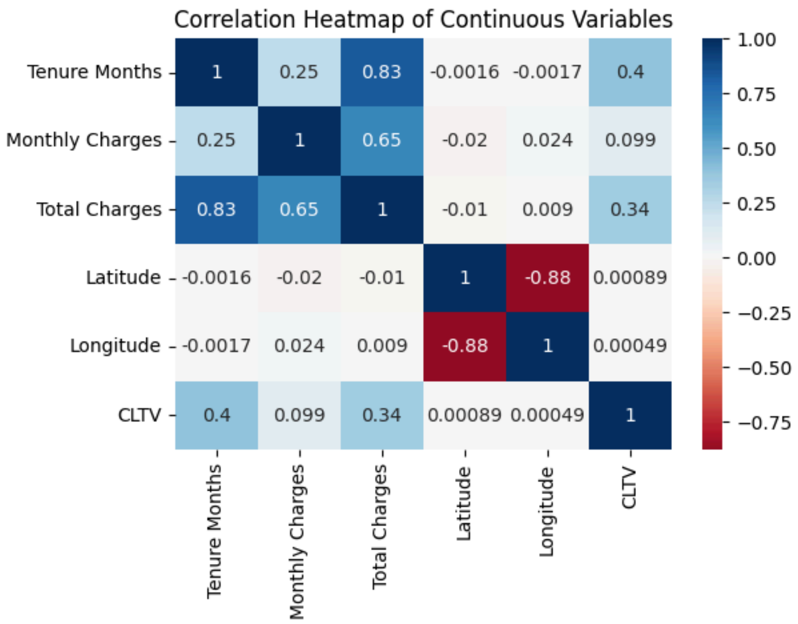
The boxplot shows that the median tenure for customers who churned is significantly lower than for those who stayed. Most customers who left did so within their first 10 to 30 months of service, suggesting that 'younger' customers are at a much higher risk of leaving.

Furthermore, customers who churned have a higher median monthly charge (roughly between 70 and 90) compared to active customers, who are more concentrated in the lower price ranges. In conclusion, both variables appear to be strong predictors of churn; high monthly charges and low customer tenure are the primary characteristics of the customers most likely to leave the service.

8.Check the correlations of the continuous variables.

```
# Convert Total Charges to numbers
df['Total Charges'] = pd.to_numeric(df['Total Charges'], errors='coerce')
# Select only the continuous columns
continuous_vars = df[['Tenure Months', 'Monthly Charges', 'Total Charges', 'Latitude', 'Longitude', 'CLTV']]

# Calculate the correlation matrix
correlation_matrix = continuous_vars.corr()
# 2. Build the Heatmap
plt.figure(figsize=(6, 4))
sns.heatmap(correlation_matrix, cmap='RdBu', center=0, annot=True)
plt.title('Correlation Heatmap of Continuous Variables')
plt.show()
```



The correlation matrix reveals that Total Charges has an extremely high positive correlation with both Tenure Months and Monthly Charges. This is expected, as Total Charges is the cumulative result of how long a customer stays and how much they pay each month. There is also a notable positive correlation between CLTV and both tenure and charges. This indicates that customers who stay longer and pay higher rates are predicted to be more valuable to the company over their lifetime. Interestingly, there is only a slight positive correlation between Tenure and Monthly Charges. This suggests that a customer’s monthly rate doesn't necessarily increase just because they have been with the company for a long time. Finally, both Latitude and Longitude show almost zero correlation with any other variables. This confirms that, in this dataset, a customer's physical location does not have a direct mathematical relationship with their billing or loyalty.

9. Check for missing values. What did you find?

```
# Check for missing values in each column
print("Missing values per column:")
print(df.isnull().sum())

# Check the total number of missing values in the DataFrame
print("\nTotal missing values in the dataset:")
print(df.isnull().sum().sum())
```

```
Missing values per column:
CustomerID           0
Count                0
Country              0
State                0
City                 0
Zip Code             0
Lat Long             0
Latitude             0
Longitude            0
Gender               0
Senior Citizen       0
Partner              0
```

```
Dependents              0
Tenure Months           0
Phone Service           0
Multiple Lines          0
Internet Service        0
Online Security         0
Online Backup           0
Device Protection       0
Tech Support            0
Streaming TV            0
Streaming Movies        0
Contract                0
Paperless Billing       0
Payment Method          0
Monthly Charges         0
Total Charges          11
Churn Label             0
Churn Value             0
Churn Score             0
CLTV                    0
Churn Reason         5174
dtype: int64

Total missing values in the dataset:
5185
```

```
# 1. Ensure Total Charges is numeric
# 'coerce' tells Python: "If you find something that isn't a number, just turn it into a NaN (Not a Number/Null)
df['Total Charges'] = pd.to_numeric(df['Total Charges'], errors='coerce')

# 2. Filter for rows where BOTH are null
# we use '&' for 'and'
both_null = df[df['Total Charges'].isnull() & df['Churn Reason'].isnull()]

# 3. Display the first 10 rows with relevant columns
both_null[['CustomerID', 'Tenure Months', 'Total Charges', 'Churn Label', 'Churn Reason']].head(5)
```

	CustomerID	Tenure Months	Total Charges	Churn Label	Churn Reason
2234	4472-LVYGI	0	NaN	No	NaN
2438	3115-CZMZD	0	NaN	No	NaN
2568	5709-LVOEQ	0	NaN	No	NaN
2667	4367-NUYAO	0	NaN	No	NaN
2856	1371-DWPAZ	0	NaN	No	NaN

The dataset contains a total of 5,185 missing values, concentrated in two columns:

- Churn Reason (5,174 missing): This high number is expected because these values represent customers who have not churned. Since they are still active users, no 'reason for leaving' exists for them.
- Total Charges (11 missing): These missing values correspond to customers with 0 months of tenure. Because they are brand-new and have not yet completed their first month of service, they have not accumulated any charges.

10. Check for outliers in each of the continuous variables. Do any of them have outliers?

```
# 1. Ensure Total Charges is numeric
df['Total Charges'] = pd.to_numeric(df['Total Charges'], errors='coerce')

# 2. Select your continuous variables
cols = ['Tenure Months', 'Monthly Charges', 'Total Charges', 'CLTV']

for col in cols:
    # Calculate Q1 (25th percentile) and Q3 (75th percentile)
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    # Define bounds for outliers
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Identify the outliers
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]

    print(f"Number of outliers in {col}: {len(outliers)}")
```

```
Number of outliers in Tenure Months: 0
Number of outliers in Monthly Charges: 0
Number of outliers in Total Charges: 0
```

```
Number of outliers in CLTV: 0
```

I conducted an outlier analysis on the continuous variables (Tenure Months, Monthly Charges, Total Charges, and CLTV) using the Interquartile Range (IQR) method. Conclusion: None of the continuous variables in this dataset contain outliers

11. Check for row outliers using Mahalanobis distance. What did you find?

```
from scipy.stats import chi2

# 1. Fill the 11 NaN values in Total Charges with 0
df['Total Charges'] = df['Total Charges'].fillna(0)

# 2. Select variables for Mahalanobis
features = df[['Tenure Months', 'Monthly Charges', 'Total Charges', 'CLTV']]

# 3. Calculate mean and covariance matrix
mean = np.mean(features, axis=0)
cov = np.cov(features.values.T)
inv_cov = np.linalg.inv(cov)

# 4. Calculate Mahalanobis Distance for each row
def mahalanobis(x):
    diff = x - mean
    return np.dot(np.dot(diff, inv_cov), diff.T)

df['Mahalanobis'] = features.apply(mahalanobis, axis=1)

# 5. Define threshold (99.9% confidence, 4 degrees of freedom)
threshold = chi2.ppf((1 - 0.001), df=4)
outliers_multi = df[df['Mahalanobis'] > threshold]

print(f"Number of multivariate outliers: {len(outliers_multi)}")
```

```
Number of multivariate outliers: 0
```

Even when looking at the four-dimensional relationship between these variables, no single customer record deviated significantly from the general pattern of the group. The result of zero outliers confirms that the continuous data is highly consistent. Every customer's total billing and predicted lifetime value align logically with their monthly rates and how long they have been with the company